

人工智能基础与应用 · 大模型实训

M2 · 分词与表示

从零实现大模型：M0-M9 知识主干（16 周 / 11 课题 / 零预算）

孙承泽 · 能动强基 2501 · 西安交通大学

2026-09-17 生成 · 版本 v1.0

Markdown 母版：人工智能基础与应用/讲义/ 下的 .md 文件 · 配套上游：CS336 A1-A5 · nanochat · minimind

讲义 M2 · 分词与表示

对应课题：课题 01 BPE 分词器 (W1) | 模块：M2 | 预计 2 个时段 (3–5 小时) 对标材料：CS336 A1 handout (规格权威) · 李沐 d2l ch14.6 子词嵌入 (中文) · HF LLM Course ch2 读法：先读下面的“要回答的问题”，再读对标材料，最后回来看本讲义主干。

〇、本讲要回答的 5 个问题

1. BPE 训练究竟在优化什么？为什么说它“不是词汇表，而是压缩器”？
2. 为什么必须从字节起步？代价是什么？
3. 训练算法为什么容易写成 $O(n^2)$ ？如何做到官方要求的 <1.5 秒？
4. 编码时为什么必须“回放合并历史”，而不能“最长匹配优先”？
5. 同一个模型，换分词器后 loss 不可比——那该用什么指标？

一、压缩视角：BPE 到底在做什么

一句话：给定词表预算 V ，BPE 反复执行“找出当前出现次数最多的相邻 token 对 $\rightarrow$ 合并成一个新 token”。每合并一次，语料的总 token 数减少一点，直到词表填满 V 。

形式化：把语料看成一串字节 $b_1 b_2 \dots b_n$ 。定义当前切分下相邻对 (x, y) 的频次 $c(x, y)$ 。每步选

$$(x^*, y^*) = \arg \max_{(x, y)} c(x, y)$$

把语料中所有该对合并为新 token xy ，并把 (x, y) 记入 merges 列表（顺序即“资历”）。

关键推论 A（压缩率）：合并的收益可以直接量化——每合并一处，总 token 数减 1。所以“BPE 训练过程”就是“贪心压缩过程”，merges 列表本质是一本压缩字典，且按收益从大到小排序。

关键推论 B（为什么 loss 不可跨分词器比较）：token 越少 $\rightarrow$ 每个 token 承载信息越多 $\rightarrow$ 单 token 的交叉熵必然更高。这是记账单位问题，不是模型好坏。所以跨分词器比较必须换成与切分无关的单位：bits-per-byte (bpb)。

这条是新课标里最重要的一条：nanochat 全程用 bpb 而不是 loss 来跨模型比较，课题 03 (scaling law) 和课题 08 (RL 对照) 都会回到这里。

二、为什么必须字节起步（以及代价）

从 Unicode 字符起步的三个具体麻烦：

麻烦	具体例子	后果
词表爆炸且长尾	仅 CJK 就有 9 万余字符，常用仅数千	大部分字符训练不充分，嵌入近乎随机
永远有未登录	新 emoji（每年新增）、生僻字、方言字	无兜底机制 $\rightarrow$ OOV，直接崩
失去字节级共享	“妈”与“吗”共享部首但完全不同字符	学不到字形/字节层面的结构

从字节起步：任何文本 $\rightarrow$ UTF-8 $\rightarrow$ 0–255 的字节序列 $\rightarrow$ 这 256 个 token 永远在词表里。所以字节级 BPE 永远不存在“无法表示的输入”，只可能被切得更碎。这就是“零风险”的含义。

代价（必须知道，因为它是第 1 题的答案关键）：

- 汉字 UTF-8 占 3 字节；英文常用词常被合并成 1 token，而中文常需 1.5–2 个 token 表示 1 个汉字（取决于词表大小与语料配比） $\rightarrow$ 中文的“每字成本”高于英文

- 序列变长 → 注意力计算量 ($O(L^2)$) 上升 → 中文推理成本更高

三、训练算法：从 $O(n^2)$ 到官方要求的 <1.5 秒

朴素想法的三个陷阱（都会导致超时或超内存）：

1. 每步重扫全语料统计相邻对 → $O(V \cdot n)$, $V=500$ 、 $n \approx$ 百万级就是灾难
2. 用字符串做合并 (`text.replace("ab", "ab_new")`) → 每次 $O(n)$ 拷贝 + 无法区分“新合并的 token”与“原始字符”
3. 把语料整体读入内存再展开成 token 列表 → 5MB 语料 + 1MB 配额的内存测试必挂

正确方向的三个要点（方向，不是实现）：

- 词频级统计而非字符级：同一种“词”（如 the）在语料里重复百万次，只需统计一次它的结构，再按词频加权
- 增量更新：一次合并只影响“包含该对的相邻位置”，其余的对频次不变 → 只需局部更新
- 优先队列/堆：快速取出当前最高频的对，而不是每步全量排序

官方参考实现 0.38 秒，“玩具实现”约 3 秒——这 8 倍差距就在上面三点里。

四、编码：回放合并历史，而不是最长匹配

编码时我们没有语料统计，只有 merges 列表（一个有序的“合并历史”）。正确做法：

把文本按 UTF-8 切成字节数组

loop:

找出当前序列中所有相邻对里，在 merges 中“排名最靠前”（资历最老）的那一对

合并它

直到没有任何相邻对出现在 merges 中

为什么不是“最长匹配优先”？因为训练时每一次合并都遵循“最老优先”的规则；若编码用另一套规则，同一个子串在不同上下文会被切成不同 token，与 tiktoken/GPT-2 的逐 id 对齐测试必然失败。BPE 的编码本质是回放训练时的决策序列。

特殊 token 是另一套优先级：它们不参与合并，且在编码时先于普通文本被匹配。注意两个官方测试点名的情况：重叠的特殊 token（如 a 与 ab 同时存在）与尾随换行 (`<|endoftext|>\n`) ——这类边界的处理顺序会直接决定对错。

五、接口设计决定内存上界（`encode` vs `encode_iterable`）

官方测试里有一个耐人寻味的设置：

方法	官方态度
<code>encode(text)</code>	被 xfail 标记：“encode 预期会超过 1MB 配额”——超了也原谅
<code>encode_iterable(iterable)</code>	用 RLIMIT_AS 硬限 1MB，读 5MB 语料——超了就失败

为什么这不是双标：接口的输入形态决定了它的内存下界。`encode(str)` 的输入本身就是一整段字符串，它已经在内存里了——上界由输入决定，实现者无能为力。`encode_iterable(iterable)` 的契约是流式：输入是迭代器，实现就必须逐块处理、边读边 yield。任何 `list(iterable)` / `"".join(iterable)` / 先聚合再编码的写法，都违背了接口语义。

这是软件工程里最漂亮的一课：接口不只是名字，它是**约束**。同一个算法，换个接口，内存上界就从“输入大小”降到“块大小”。这个思想在课题 05（KV cache）、课题 07（数据流式清洗）、课题 10（长程 Agent 的记忆压缩）都会反复出现。

六、词表大小的三方权衡（第 1、4 题的答案框架）

词表从 V 扩到 4V，同时影响三件事：

受影响项	方向	机理
嵌入/输出层参数量	↑ 约 4 倍	参数 $\propto V \cdot d$ （若权重共享则 1 份，否则 2 份）
同文本序列长度 L	↓	更多常见片段被合并 $\rightarrow$ token 数下降
每 token 的注意力计算	↓ 约 L^2 下降	注意力是 $O(L^2 \cdot d)$
罕见 token 的训练充分度	↓	长尾 token 出现次数少，嵌入学不好

净效果**不是单调的**：太小 $\rightarrow$ 序列太长、算力烧在长度上；太大 $\rightarrow$ 参数浪费 + 长尾欠训练。经验上 32k–128k 是常见区间（GPT-2 用 50257，现代模型多在 100k 量级，且对多语言友好）。

中文特例：词表若中文覆盖不足，一个汉字可能被拆成 3 个字节 token，**中文成本直接翻 2–3 倍**——这是国产模型普遍重视中文词表预算的原因。

七、常见误解（每条配反例）

误解	反例
“一个 token 就是一个词”	<code>_transformer</code> 可能是 1 个 token，妈 可能是 3 个 token（UTF-8 三字节）
“分词只是预处理，不影响模型能力”	词表决定序列长度 $\rightarrow$ 决定有效上下文与算力分配 $\rightarrow$ 直接影响能学到什么
“编码用最匹配更聪明”	会与训练决策不一致 $\rightarrow$ 与 tiktoken 逐 id 对齐测试直接失败
“词表越大越好”	长尾 token 训练不足 + 嵌入层参数浪费；存在经验最优
“decode 一定无损”	只有在 完整 utf-8 序列 上才无损；单 token 解码可能是半个字符 (👤)

八、能立刻跑的实验（讲完就做，知行咬合点）

```
# ① 跑判分基线（现在应当是 27 failed + 1 xfailed, 全部因缺失手写文件）
环境/judge/judge.sh 01 --quick

# ② 看官方参考 merges 长什么样（前 20 条）——体会“资历顺序”的直觉
head -20 上游/assignment1-basics/tests/fixtures/train-bpe-reference-merges.txt

# ③ 亲眼看字节：一个汉字到底占几个 token（用官方 fixture 的 GPT-2 词表）
环境/.venv/bin/python - <<'PY'
import tiktoken, os
os.environ["TIKTOKEN_CACHE_DIR"] = "环境/tiktoken-cache"
enc = tiktoken.get_encoding("gpt2")
for s in ["hello world", "你好", "你好，世界！", "👤"]:
    ids = enc.encode(s)
    print(f"{s!r:16}  $\rightarrow$  {len(ids)} 个 token {enc.decode([i] for i in ids)}")
print("> 汉字/emoji 被切成多个字节 token 的现象，一眼可见")
PY
```

九、代码级提问（3-5 道，先写预测再动手）

1. **【改坏预测】** 把训练里“并列最高频时任意取一个”改成“取字节序最小的一对”，`test_train_bpe` 会过吗？`test_train_bpe_speed` 呢？**先写预测和理由。**
2. **【诊断】** 你的实现通过了往返一致性测试，但 `test_train_bpe` 的 merges 比对失败。请按顺序列出你会检查的 4 件事（提示方向：预处理、并列规则、特殊 token、词表初值）。
3. **【代价估算】** 同一句话，词表从 32k 换到 128k：① 嵌入参数量变几倍 ② 序列长度大致变几成 ③ **bpb** 会变吗？为什么？
4. **【接口】** 若把 `encode_iterable` 写成“先 `list(iterable)` 再拼接”，哪个测试会挂？挂的形式是断言失败，还是进程被直接杀掉？（想清楚 `RLIMIT_AS` 的作用方式）
5. **【迁移】** 课题 05 要写 KV cache，它会遇到和“`encode` vs `encode_iterable`”同构的内存问题吗？说说你的类比。

十、记忆锚

- 隐喻：BPE = “贪心的拼音缩写机”——不停找最常一起出现的两段，把它们缩成一个新记号，缩到最后词表满了；字典按先来后到排资历。
- 口诀：“字节兜底、频次贪心、老手优先、流式省内存”（对应：字节起步 / 训练目标 / 编码回放 / 接口约束）
- 符号位置法： V （词表）在上，撑着两件事：下面是 参数量 $\propto V \cdot d$ ，右边是 序列长度 $L \downarrow$ ；三者写成一个三角形，随时能想起“扩词表 = 拿参数换长度”。

十一、本讲与后续的接口（预告）

- **M3:** 序列长度 L 怎么进入注意力计算量 ($O(L^2)$) —— 词表选择的代价在这一讲兑现
- **M4:** bpb 作为跨规模比较单位——scaling law 的纵轴
- **M5:** `encode_iterable` 的流式思想 → KV cache 的“不重算历史”
- **课程板块:** 本讲直接对应课程大纲的“大语言模型”入门，也是大论文“方法”章的第一段素材